

Understanding and Configuring Layered Security in an AWS VPC

Lab Details

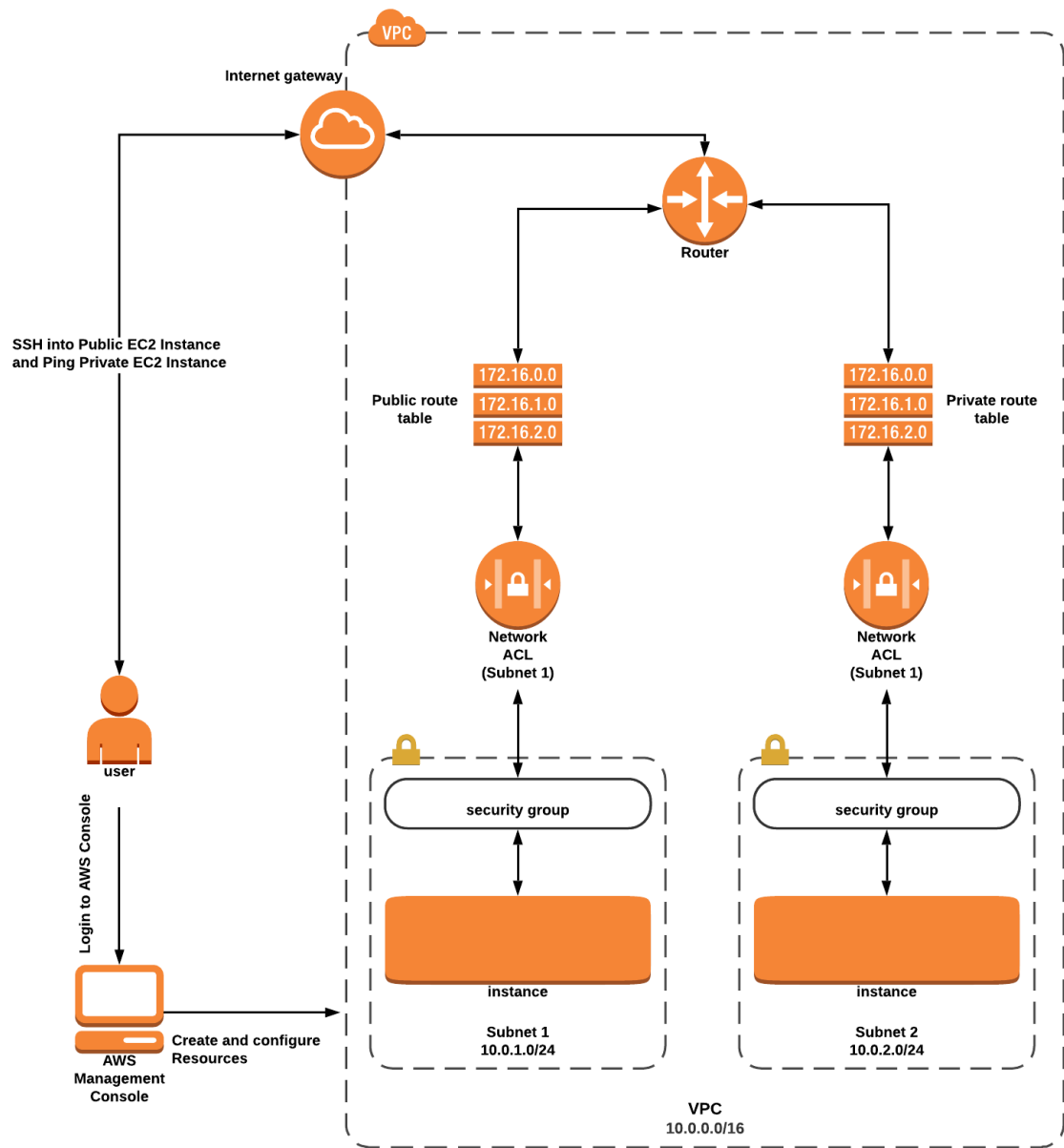
1. This lab walks you through the steps to Configure Multi-layered Security in AWS VPC and to launch 2 EC2 instances (one in a public subnet and another in a private subnet)
2. You will practice it using Amazon VPC, Amazon EC2
3. Duration: **1 Hour**
4. AWS Region: **US East (N. Virginia) us-east-1**

Introduction

Amazon Virtual Private Cloud

- Amazon VPC allows us to launch **AWS resources in an isolated network** that is defined by us in a **more private and secure environment**.
- This feature enables us to **increase the security level** of the AWS resources.
- The AWS resources can be protected using **multilayered VPC** which includes **security groups** and **Network Access Control list**.
- The VPC **security group** provides security at **instance level** which acts like a firewall and controls both inbound and outbound traffic.
- The VPC **NACL** provides security at **Network Level** i.e **subnet level** which acts like a firewall for associated subnets and controls inbound and outbound traffic.

Architecture Diagram



Task Details

1. In this lab, we are going to **Configure Multi layered VPC security and launch 2 AWS EC2 instances.**
2. Create a **VPC** with your **custom CIDR block IP range.**
3. Create **2 subnets** for **public** and **private** resources. AWS resources in the public subnet will have access to the internet, while AWS resources in the private subnet will be protected and isolated from the internet.

4. Create an **Internet gateway** and **associate it with the VPC**. This is needed in order for the resources to connect to the external world.
5. Create **2 Route tables** (one for public routing and one for private) and then **provide the routes** and associate the **subnets accordingly**.
6. Launch **2 AWS EC2 instances** (one in the public subnet and one in the private subnet).
7. SSH into the public EC2 Instance and use the **ping command**, try to reach the **Private IP** of the private EC2 instance.
8. **Verify the response** of the ping command to know whether the instance is reachable from the internet.

Lab Steps

Note: There is no Validation function for this lab.

Task 1: Create a Custom VPC

1. Make sure to choose the **N.Virginia** region in the AWS Management Control dashboard, which is present in the top right corner.
2. Navigate and click on **VPC** which will be available under the  **Networking & Content Delivery** section of  **All services**
3. Click on **Your VPCs** from the left menu.
4. In **Your VPCs** page fill the following details,
 - Name tag: Enter **whizlabs_VPC**
 - IPV4 CIDR Block: Enter **10.0.0.0/16**
 - IPV6 CIDR block: Select **No IPV6 CIDR block**
 - Tenancy: **Default**
 - Click on **Create VPC** button.

VPC settings

Name tag - *optional*
Creates a tag with a key of 'Name' and a value that you specify.

IPv4 CIDR block [Info](#)

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - <i>optional</i>	
Name	whizlabs_VPC	Remove

You can add 49 more tags.

Note: Kindly note that there will be a Default VPC. Please don't edit or delete it.

Task 2: Creating the Internet gateway

1. You need to create an **Internet Gateway** in order for the resources to connect to the external world. **Go to Services--> VPC-->Internet Gateways-->** Click on **Create Internet gateway** and provide the following details:
 - Name tag: Enter **whizlabs_IGW**
 - Click on **Create internet gateway** button.

Create internet gateway Info

An internet gateway is a virtual router that connects a VPC to the internet. To create a new internet gateway specify the name for the gateway below.

Internet gateway settings

Name tag
Creates a tag with a key of 'Name' and a value that you specify.

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	whizlabs_IGW	Remove

You can add 49 more tags.

- Next we will **attach the created IGW to the VPC**. To do so, **Select** the created Internet gateway --> **Go to Actions** --> **Attach to VPC** --> Select the appropriate VPC from the dropdown and then click on **Attach Internet gateway**.

Attach to VPC (igw-0af32934a42f2971a) Info

VPC
Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.

Available VPCs
Attach the internet gateway to this VPC.

vpc-0c3e0e56765905116 - test
vpc-0fced3cdddbdc8694 - whizlabs_VPC

Task 3: Creating subnets

- Create **2 Subnets**, one for public and another for private resources. First, we will create the **Public subnet**.

- Go to **Services --> VPC --> Subnets --> Create Subnets** and provide the following details:
 - VPC ID: Select **whizlabs_VPC** (Select the VPC which you created from the dropdown)
 - Name tag: Enter **public_subnet**
 - Availability zone: Select **No Preference**
 - IPv4 CIDR block: Enter **10.0.1.0/24**
 - Click on **Create subnet** button
- Similarly, create the **Private subnet**. Go to **Services --> VPC --> Subnets --> Create Subnets** and provide the following details:
 - VPC ID: Select **whizlabs_VPC** (Select the VPC which you created from the dropdown)
 - Name tag: Enter **private_subnet**
 - Availability zone: Select **No Preference**
 - IPv4 CIDR block: Enter **10.0.2.0/24**
 - Click on **Create subnet** button

Subnets (8) [Info](#)

☐	Name ▾	Subnet ID ▾	State ▾	VPC ▾
☐	–	subnet-65a2d94b	✔ Available	vpc-0de26a77 Default VPC
☐	–	subnet-ae048490	✔ Available	vpc-0de26a77 Default VPC
☐	public_subnet	subnet-064a24df0777108eb	✔ Available	vpc-012a0b04dece2bb2c whizlabs_VPC
☐	private_subnet	subnet-014d04281bfdc38c0	✔ Available	vpc-012a0b04dece2bb2c whizlabs_VPC
☐	–	subnet-26acd57a	✔ Available	vpc-0de26a77 Default VPC
☐	–	subnet-2e4b3749	✔ Available	vpc-0de26a77 Default VPC
☐	–	subnet-06d9d54c	✔ Available	vpc-0de26a77 Default VPC
☐	–	subnet-fce7d1f3	✔ Available	vpc-0de26a77 Default VPC

Note: There will be 6 default subnets available in the N.Virginia region. Please don't delete or edit these subnets.

Task 4: Creating Route tables, configuring routes and associating subnets

- Create **2 route tables**, one for public routes and another for private routes.
- Create a **public route table**. Go to **Services → VPC → Route Tables → Create Route Tables** and provide the following details:
 - Name tag: Enter **public_route**

- VPC: Select **whizlabs_VPC** (Select the VPC you created from the dropdown)
- Click on **Create route table** button.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

VPC
The VPC to use for this route table.

vpc-012a0b04dece2bb2c (whizlabs_VPC)
vpc-0de26a77 (Default VPC) (default)

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key

Value - optional

You can add 49 more tags.

3. Similarly, create a **private route table**, **Go to Services** → **VPC** → **Route Tables** → **Create Route Tables** and provide the following details:
 - Name tag: Enter **private_route**
 - VPC: Select **whizlabs_VPC** (Select the VPC you created from the dropdown)
 - Click on **Create route table** button.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

private_route

VPC
The VPC to use for this route table.

vpc-012a0b04dece2bb2c (whizlabs_VPC)

Q |

vpc-012a0b04dece2bb2c (whizlabs_VPC)

vpc-0de26a77 (Default VPC) (default)

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

Q Name X Q private_route X Remove

Add new tag

You can add 49 more tags.

Cancel Create route table

Note: Don't delete or make changes to the default route tables present in the account.

- You need to **add routes to the Route Tables**. Select **Public_Route_Table** and then click on **Routes** in the bottom window --> Click on **Edit Routes** --> Click on **Add Route** --> **Destination** → **0.0.0.0/0** and in **Target** → **whizlabs_IGW** select the Internet Gateway and choose our Internet Gateway (leave the existing route). Click **Save Routes**.

Edit routes

Destination	Target	Status	Propagated
10.0.0.0/16	Q local X	Active	No
Q 0.0.0.0/0 X	Q igw- X igw-08689bd74d1fc7425 (whizlabs_IGW)	-	No Remove

Add route

Cancel Preview Save changes

- Next we will associate the **Public subnet** with this **Public_Route_Table**. Select **Public_Route_Table** --> Click on the **Subnets associations** Tab in the bottom

window --> Click on **Edit subnet Association** --> **Select the public_subnet** --> Click on **Save**.

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/2)

Q Filter subnet associations

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☒	public_subnet	subnet-064a24df0777108eb	10.0.1.0/24	–	Main (rtb-01b32f040fec02851)
☐	private_subnet	subnet-014d04281bfdc38c0	10.0.2.0/24	–	Main (rtb-01b32f040fec02851)

Selected subnets

subnet-064a24df0777108eb / public_subnet X

Cancel Save associations

- Similarly, **Select Private_Route_Table** --> Click on the **Subnets associations** Tab in the bottom window --> Click on **Edit subnet Association** --> **Select the private_subnet** --> Click on **Save**.

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/2)

Q Filter subnet associations

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☐	public_subnet	subnet-064a24df0777108eb	10.0.1.0/24	–	rtb-0285c319a3c53fc18 / public_route
☒	private_subnet	subnet-014d04281bfdc38c0	10.0.2.0/24	–	Main (rtb-01b32f040fec02851)

Selected subnets

subnet-014d04281bfdc38c0 / private_subnet X

Cancel Save associations

Note: We haven't created any routes for private_subnet. Please don't edit or delete the default Route table.

Task 5: Creating Security Groups

- Create a **security group** which will provide security at the instance level. **Go to Services → VPC → Security** → Click on **Security Groups** → Click on **Create Security Group** and then provide the following details:
 - Security group name: Enter **whizlabs_securitygroup**
 - Description: Enter **security group for multilayered VPC**
 - VPC: Select **whizlabs_VPC** (select from the dropdown)
 - Under **Inbound Rules**, click on **Add Rule**.
 - To add **SSH**,

- Choose Type: SSH
 - Source: **Custom - 0.0.0.0/0**
- To add **All ICMP - IPv4**,
 - Click on **Add Rule**
 - Choose Type: **All ICMP - IPv4**
 - Source: **Custom - 0.0.0.0/0**
- Click on **Create security group**

Create security group Info

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name Info

Name cannot be edited after creation.

Description Info

VPC Info

Inbound rules Info

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
SSH	TCP	22	Custom 0.0.0.0/0		Delete
All ICMP - IPv4	ICMP	All	Custom 0.0.0.0/0		Delete

Add rule

Note: Please don't delete or edit the other security groups present in your account.

Task 6: Creating a NACL, Configuring Inbound Rules and Associating Subnets

- We will start by creating a **NACL**. To do so go to **Go to Services** → **VPC** → **Network ACLs** and click on **Create Network ACL** Provide the following details:

- Name tag: Enter **whizlabs_NACL**
- VPC: Select **whizlabs_VPC** (Select the VPC which you created from the dropdown)
- Click on **Create network ACL** button.

Network ACL settings

Name - optional
Creates a tag with a key of 'Name' and a value that you specify.

VPC
VPC to use for this network ACL.

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	whizlabs_NACL	Remove

You can add 49 more tags.

2. Select **whizlabs_NACL** and then in the bottom window click on **Inbound Rules** → **Edit Inbound Rules** and then click on the **Add Rule**.
3. Add the following rules:

- For **SSH**, click on ,
 - Rule# : Enter **100**
 - Type: Choose **SSH (22)**
 - Source: Enter **0.0.0.0/0**
 - Allow / Deny: Select **Allow**
- For **ALL ICMP- IPv4**, click on ,
 - Rule# : Enter **200**
 - Type: Choose **ALL ICMP - IPv4**

- Source: Enter **0.0.0.0/0**
- Allow / Deny: Select **Allow**
- Click on **Save changes**.

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the VPC.

Rule number Info	Type Info	Protocol Info	Port range Info	Source Info	Allow/Deny Info	
100	SSH (22)	TCP (6)	22	0.0.0.0/0	Allow	Remove
200	Custom ICMP - IPv4	All	N/A	0.0.0.0/0	Allow	Remove
*	All traffic	All	All	0.0.0.0/0	Deny	

[Add new rule](#) [Sort by rule number](#)

[Cancel](#) [Preview changes](#) [Save changes](#)

4. NACLs are stateless. You need to add the same rule in Outbound rules too. To do so click on Outbound Rules→ Edit Outbound Rules→ Add Rule and then provide the following details:

- For **SSH**, click on [Add Rule](#) ,
 - Rule# : Enter **100**
 - Type: Choose **SSH (22)**
 - Destination: Enter **0.0.0.0/0**
 - Allow / Deny: Select **Allow**
- For **ALL ICMP- IPv4**, click on [Add Rule](#) ,
 - Rule# : Enter **200**
 - Type: Choose **ALL ICMP - IPv4**
 - Destination: Enter **0.0.0.0/0**
 - Allow / Deny: Select **Allow**
- For **Custom TCP Rule**, click on [Add Rule](#) ,
 - Rule# : Enter **300**
 - Type: Choose **Custom TCP Rule**
 - Port Range: Enter **1024 - 65535**
 - Destination: Enter **0.0.0.0/0**
 - Allow / Deny: Select **Allow**
- Click on **Save changes**.

Edit outbound rules Info

Outbound rules control the outgoing traffic that's allowed to leave the VPC.

Rule number Info	Type Info	Protocol Info	Port range Info	Destination Info	Allow/Deny Info	
100	SSH (22)	TCP (6)	22	0.0.0.0/0	Allow	Remove
200	Custom ICMP - IPv4	All	N/A	0.0.0.0/0	Allow	Remove
300	Custom TCP	TCP (6)	1024 - 65535	0.0.0.0/0	Allow	Remove
*	All traffic	All	All	0.0.0.0/0	Deny	

5. **Associate both public and private subnets with this NACL.** To do so, **Select the whizlabs_NACL** → Click on **Subnet Association** in the bottom window → **Edit the Subnet association** → **Select both subnets** we created and click on **Edit**.

Edit subnet associations Info

Change which subnets are associated with this network ACL.

Available subnets (2/2)

Filter subnet associations

☒	Name	Subnet ID	Associated with	Availability zone	IPv4 CIDR	IPv6 CIDR
☒	public_subnet	subnet-064a24df0777108eb	acl-0e28dbf943b942526	us-east-1e	10.0.1.0/24	-
☒	private_subnet	subnet-014d04281bfcd38c0	acl-0e28dbf943b942526	us-east-1e	10.0.2.0/24	-

Selected subnets

Task 7: Launching 2 EC2 Instances

1. Navigate to **All services** and choose **EC2** under **Compute**
2. Navigate to **Instances** on the left panel and click on **Launch instances**.
3. On the next page, choose an Amazon Machine Image (AMI): Search for **Amazon Linux AMI** in the search box and click on the select button.

Step 1: Choose an Amazon Machine Image (AMI) Cancel and Exit

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. You can select an AMI provided by AWS, our user community, or the AWS Marketplace, or you can select one of your own AMIs.

Q, Amazon Linux AMI 2018

Search by Systems Manager parameter

Quick Start (1)

My AMIs (0)

AWS Marketplace (4)

Community AMIs (2457)

Amazon Linux **Free tier eligible** **Amazon Linux AMI 2018.03.0 (HVM), SSD Volume Type** ami-09d8b5222f2b93bf0

The Amazon Linux AMI is an EBS-backed, AWS-supported image. The default image includes AWS command line tools, Python, Ruby, Perl, and Java. The repositories include Docker, PHP, MySQL, PostgreSQL, and other packages.

Root device type: ebs Virtualization type: hvm ENA Enabled: Yes

64-bit (x86)

- Choose instance type as **t2.Micro** and click on **Configuration Instance Details**.
- In the **Configure Instance details** make changes to the following parameters:
 - Network: Select **whizlabs_VPC** (select from dropdown)
 - Subnet: Select **public_subnet** (select from dropdown)
 - Auto-assign Public IP: Select **Enable** (select from dropdown)
 - Leave all values as **Default** and then click on **Add Storage**.

1. Choose AMI 2. Choose Instance Type 3. Configure Instance 4. Add Storage 5. Add Tags 6. Configure Security Group 7. Review

Step 3: Configure Instance Details

Configure the instance to suit your requirements. You can launch multiple instances from the same AMI, request Spot instances to take advantage of the lower pricing, assign an access management role to the instance, and more.

Number of instances [Launch into Auto Scaling Group](#)

Purchasing option ☐ Request Spot instances

Network [Create new VPC](#)

Subnet [Create new subnet](#)

Auto-assign Public IP

Placement group ☐ Add instance to placement group

Capacity Reservation [Create new Capacity Reservation](#)

IAM role [Create new IAM role](#)

Shutdown behavior

Enable termination protection ☐ Protect against accidental termination

Monitoring ☐ Enable CloudWatch detailed monitoring

[Cancel](#) [Previous](#) [Review and Launch](#) [Next: Add Storage](#)

- On the **Add Storage** page, leave default and click on **Next: Add Tags**.
- Create a **Tag** with the name **public_instance** and then click on **Configure Security Group**.

1. Choose AMI 2. Choose Instance Type 3. Configure Instance 4. Add Storage 5. Add Tags 6. Configure Security Group 7. Review

Step 5: Add Tags

A tag consists of a case-sensitive key-value pair. For example, you could define a tag with key = Name and value = Webserver. A copy of a tag can be applied to volumes, instances or both. Tags will be applied to all instances and volumes. [Learn more](#) about tagging your Amazon EC2 resources.

Key (128 characters maximum)	Value (256 characters maximum)	Instances	Volumes
Name	public_instance	☒	☒

[Add another tag](#) (Up to 50 tags maximum)

- In **Configure Security group** → **Select an existing security group** and then select → **whizlabs_securitygroup** and click on **Review and Launch**.

1. Choose AMI 2. Choose Instance Type 3. Configure Instance 4. Add Storage 5. Add Tags 6. Configure Security Group 7. Review

Step 6: Configure Security Group

A security group is a set of firewall rules that control the traffic for your instance. On this page, you can add rules to allow specific traffic to reach your instance. For example, if you want to set up a web server and allow Internet traffic to reach your instance, add rules that allow unrestricted access to the HTTP and HTTPS ports. You can create a new security group or select from an existing one below. [Learn more](#) about Amazon EC2 security groups.

Assign a security group: ☐ Create a new security group ☒ Select an existing security group

Security Group ID	Name	Description	Actions
sg-07f2b0db04bdb42	default	default VPC security group	Copy to new
sg-0a21a09867e045e47	whizlabs_securitygroup	security group for multilayered VPC	Copy to new

9. Review the instance setting and then click on the **Launch. Create a new key pair** with the name ***whizlabs.pem*** and download and save the keypair. Lastly, click on **Launch Instances**.

Select an existing key pair or create a new key pair ×

A key pair consists of a **public key** that AWS stores, and a **private key file** that you store. Together, they allow you to connect to your instance securely. For Windows AMIs, the private key file is required to obtain the password used to log into your instance. For Linux AMIs, the private key file allows you to securely SSH into your instance.

Note: The selected key pair will be added to the set of keys authorized for this instance. Learn more about [removing existing key pairs from a public AMI](#).

Create a new key pair

Key pair name

whizlabs.pem

Download Key Pair

You have to download the **private key file** (*.pem file) before you can continue. Store it in a **secure and accessible location**. You will not be able to download the file again after it's created.

Cancel

Launch Instances

10. Similar to the above, launch **another EC2 instance** choosing the **same VPC**. Choose **Private subnet** and Name the instance as ***private_instance*** and add the EC2 to **whizlabs_securitygroup**.
11. **Launch the instance with the same key pair.**

Network ⓘ

vpc-0fced3cdddbdc8694 | whizlabs_VPC

Create new VPC

Subnet ⓘ

subnet-0ad48cd7479ad3ddd | private_subnet | us-east-1

Create new subnet

Auto-assign Public IP ⓘ

Use subnet setting (Disable)

Step 5: Add Tags

A tag consists of a case-sensitive key-value pair. For example, you could define a tag with key = Name and value = Webserver.
A copy of a tag can be applied to volumes, instances or both.
Tags will be applied to all instances and volumes. [Learn more](#) about tagging your Amazon EC2 resources.

Key (128 characters maximum)	Value (256 characters maximum)	Instances ⓘ	Volumes ⓘ
Name	private_instance	☒	☒

[Add another tag](#) (Up to 50 tags maximum)

Select an existing key pair or create a new key pair

A key pair consists of a **public key** that AWS stores, and a **private key file** that you store. Together, they allow you to connect to your instance securely. For Windows AMIs, the private key file is required to obtain the password used to log into your instance. For Linux AMIs, the private key file allows you to securely SSH into your instance.

Note: The selected key pair will be added to the set of keys authorized for this instance. [Learn more about removing existing key pairs from a public AMI.](#)

Select a key pair

☒ I acknowledge that I have access to the selected private key file (whizlabs.pem.pem), and that without this file, I won't be able to log into my instance.

[Cancel](#)
[Launch Instances](#)

Task 8: Testing the newly-launched EC2 instances

1. Get the **IPv4 public IP** of the public_instance from the EC2 dashboard. To do so select public_instance and look in the **Details** tab.

	Name ▾	Instance ID	Instance state ▾	Instance type ▾
☒	public_instance	i-06ea7f02744ad1526	Running	t2.micro
☐	private_instance	i-0d6707cd5b9184556	Running	t2.micro

Instance: i-06ea7f02744ad1526 (public_instance)

Details	Security	Networking	Storage	Status checks	Monitoring	Tags
---------	----------	------------	---------	---------------	------------	------

▼ Instance summary [Info](#)

Instance ID i-06ea7f02744ad1526 (public_instance)	Public IPv4 address 100.25.36.149 open address
--	---

- Repeat the steps above to get the **IPv4 Private IP** of private_instance.
- SSH into EC2 Instance using **public_instance**
 - Please follow the steps in [SSH into EC2 Instance](#).
- Ping** to the **private IP** of your **private_instance** by using the below command:
 - `ping < your Private EC2 IP address >`
- Once you execute this command, you will receive a response from the IP.

```
ec2-user@ip-10-0-1-106:~  
https://aws.amazon.com/amazon-linux-ami/2018.03-release-notes/  
[ec2-user@ip-10-0-1-106 ~]$  
login as: ec2-user  
Authenticating with public key "imported-openssh-key"  
Last login: Thu Aug  6 10:27:19 2020 from 160.238.74.74  
  
  _ | _ | _ )  
  _ | ( _ | /  Amazon Linux AMI  
  _ | \ _ | _ |  
  
https://aws.amazon.com/amazon-linux-ami/2018.03-release-notes/  
[ec2-user@ip-10-0-1-106 ~]$ ping 10.0.2.212  
PING 10.0.2.212 (10.0.2.212) 56(84) bytes of data.  
64 bytes from 10.0.2.212: icmp_seq=1 ttl=255 time=0.357 ms  
64 bytes from 10.0.2.212: icmp_seq=2 ttl=255 time=0.410 ms  
64 bytes from 10.0.2.212: icmp_seq=3 ttl=255 time=0.463 ms  
64 bytes from 10.0.2.212: icmp_seq=4 ttl=255 time=0.443 ms  
64 bytes from 10.0.2.212: icmp_seq=5 ttl=255 time=0.444 ms  
64 bytes from 10.0.2.212: icmp_seq=6 ttl=255 time=0.486 ms  
64 bytes from 10.0.2.212: icmp_seq=7 ttl=255 time=0.449 ms  
^C  
--- 10.0.2.212 ping statistics ---  
7 packets transmitted, 7 received, 0% packet loss, time 6128ms  
rtt min/avg/max/mdev = 0.357/0.436/0.486/0.038 ms  
[ec2-user@ip-10-0-1-106 ~]$
```

6. Press [Ctrl] + C again to cancel the process..
7. **Note:** You were able to do these tasks because we added NACL Rules.

Completion and Conclusion

1. You have successfully used the AWS management console to Configure Multi layered VPC security.
2. You have successfully created a VPC with your custom CIDR block IP range.
3. You have successfully created 2 subnets for public and private AWS instances.
4. You have successfully created an Internet gateway and associated it with the VPC.
5. You have successfully created 2 Route tables (one public, one private), provided the routes and associated the subnets with the route tables.
6. You have successfully created a Network ACL.
7. You have successfully launched 2 EC2 instances (one in a public subnet and one in a private subnet)
8. You have successfully tested the EC2 instance for connectivity to the outside world.